ALPE FORMACIÓN · SANTANDER

Edición 2026 · PHP 8.x

CURSO DE PHP · CLASE 32

Eloquent ORM

- Modelos y convenciones
- Leer con Eloquent
- Crear, modificar y borrar
- Relaciones
- Carga ansiosa y el problema N+1
- Scopes
- Casts, accessors y mutators
- Borrado lógico
- Paginación
- La academia con Eloquent



Ramón Abramo

+25 años de experiencia docente · Santander, Cantabria

ALPE

FORMACIÓN PROFESIONAL

🧭 Contenido de la clase

En la clase 25 escribimos un `Modelo` base al estilo Active Record: `Curso::todos()`, `Curso::buscar($id)`, `$curso->guardar()`. Os dije entonces que era una versión de juguete de **Eloquent**, el ORM de Laravel. Hoy conocemos el de verdad.

Un **ORM** (*Object-Relational Mapper*) convierte filas de la base de datos en objetos y al revés. Con Eloquent, cada tabla tiene su **modelo**, cada fila es un **objeto** y las relaciones entre tablas se recorren como propiedades: `$curso->categoria->nombre`, `$alumno->cursos`. Se acabó escribir `join` a mano. 🎉

•
👌 Cómo vamos a trabajar

1
📘 Modelos y convenciones

2
👍 Leer con Eloquent

3
🚀 Crear, modificar y borrar

4
😮 Relaciones

5
⚠️ Carga ansiosa y el problema N+1

6
🔍 Scopes

7
📘 Casts, accessors y mutators

8
🤔 Borrado lógico

9
👌 Paginación

10
🚀 La academia con Eloquent

11
🚀 Ejercicios

12
👋 Lo que tenéis que llevaros de hoy



CLASE 32

👌 Cómo vamos a trabajar

Copiad el contenido de `ejemplos/` encima del proyecto y ejecutad:

terminal — php-paso-a-paso/clase32 — Visual Studio Code

terminal

```
php artisan migrate:fresh --seed
```

Esquema

```
app/Models/                    ← Categoria, Curso y Alumno completos; NUEVO Perfil; User
con su perfil
database/migrations/           ← NUEVAS: borrado lógico de cursos y tabla de perfiles
database/seeders/              ← matrículas con attach() y el perfil del profesor
app/Http/Controllers/          ← CursoController y PaginaController con Eloquent; NUEVO
CategoriaController
app/Providers/AppServiceProvider.php ← avisa del problema N+1 y configura la paginación
lang/es/pagination.php         ← NUEVO: «Anterior» y «Siguiente»
resources/views/               ← tarjeta, listado y ficha del curso; NUEVA página de
categoría
public/css/academia.css        ← estilos de la paginación
routes/web.php y api.php
```

Para probar lo que vamos viendo, tened abierta una consola con `php artisan tinker`.



CLASE 32 · PUNTO 1

📘 Modelos y convenciones

Un modelo es una clase que hereda de `Model` y vive en `app/Models/` [1]. Gracias a las **convenciones**, casi no hay que configurar nada:

Convención	Ejemplo	Si no se cumple…
Modelo en singular y `PascalCase`	`Curso`	
Tabla en plural (reglas del inglés: añade «s»)	`cursos`	`#[Table('perfiles')]`
Clave primaria `id`	`id`	`#[Table(key: 'codigo')]`
Columnas `created_at` y `updated_at`	`$table->timestamps()`	`#[WithoutTimestamps]`

¿Dónde falla la regla de la «s»? En las palabras que hacen el plural con «-es»: `Perfil` → `perfils`, `Profesor` → `profesors`. 🙃 Laravel 13 lo resuelve con un **atributo de PHP** encima de la clase (`Perfil.php`):

ejemplo.php — php-paso-a-paso/clase32 — Visual Studio Code

ejemplo.php

```php
// Sin #[Table], Laravel buscaría la tabla "perfils"
#[Table('perfiles')]
#[Fillable(['ciudad', 'biografia'])]
class Perfil extends Model
{
    public function user(): BelongsTo
    {
        return $this->belongsTo(User::class);
    }
}
```



💡 En la clase 31 lo hicimos con una propiedad, `protected $table = 'profesores';`, y así lo veréis en muchos tutoriales. Las dos formas funcionan; los atributos (`#[...]`) son la novedad de Laravel 13 y es lo que trae ya el modelo `User`.

La asignación masiva: #[Fillable]

Para crear o modificar un registro **con un array** (lo que haremos con los formularios), Eloquent exige que digamos qué columnas se pueden rellenar así (`Curso.php`):

ejemplo.php — php-paso-a-paso/clase32 — Visual Studio Code

ejemplo.php

```php
// Las columnas que se pueden rellenar de golpe con create() o update()
#[Fillable([
    'categoria_id', 'titulo', 'descripcion', 'imagen', 'horas',
    'precio', 'plazas', 'fecha_inicio', 'publicado',
])]
class Curso extends Model
```

¿Por qué? Imaginad que en la tabla `users` hubiera una columna `es_admin`. Si hiciéramos `User::create($request->all())`, un usuario listillo podría añadir `es_admin=1` al formulario con las herra-

mientas del navegador. Con `Fillable`, lo que no está en la lista **se ignora**. (La versión clásica es la propiedad `protected $fillable = [...]`.)



CLASE 32 · PUNTO 2

👍 Leer con Eloquent

Un modelo **es** un Query Builder con superpoderes: todo lo de la clase 31 (`where`, `orderBy`, `count`...) funciona igual, pero lo que devuelve son **objetos del modelo** [2].

ejemplo.php — php-paso-a-paso/clase32 — Visual Studio Code

ejemplo.php

```php
use App\Models\Curso;

Curso::all();                              // todos (una colección de objetos Curso)
Curso::find(3);                            // el de id 3, o null
Curso::findOrFail(3);                      // el de id 3, o un 404
Curso::where('horas', '>', 40)->get();
Curso::where('slug', 'php-desde-cero')->first();
Curso::orderByDesc('precio')->take(3)->get();
Curso::count();
Curso::avg('precio');
Curso::pluck('titulo', 'id');              // ['1' => 'PHP desde cero', ...]
```

Y cada objeto tiene las columnas como **propiedades**:

ejemplo.php — php-paso-a-paso/clase32 — Visual Studio Code

ejemplo.php

```php
$curso = Curso::find(3);
$curso->titulo;        // "Laravel profesional"
$curso->precio;        // "390.00"
$curso->toArray();     // todo como array
$curso->toJson();      // todo como JSON
```



CLASE 32 · PUNTO 3

🚀 Crear, modificar y borrar

ejemplo.php — php-paso-a-paso/clase32 — Visual Studio Code

ejemplo.php

```php
// Crear: opción 1, como en nuestro MVC
$categoria = new Categoria();
$categoria->nombre = 'Idiomas';
$categoria->slug = 'idiomas';
$categoria->save();

// Crear: opción 2, con un array (necesita Fillable)
$curso = Curso::create([
    'categoria_id' => 1,
    'titulo' => 'Vue.js desde cero',
    'horas' => 40,
    'precio' => 200,
]);

// Modificar
$curso->precio = 220;
$curso->save();
// o de golpe:
$curso->update(['precio' => 220, 'plazas' => 15]);

// Borrar
$curso->delete();
Curso::destroy([4, 5]);

// Buscar o crear
$categoria = Categoria::firstOrCreate(['slug' => 'idiomas'], ['nombre' => 'Idiomas']);
```

Fijaos en que no hemos escrito `created_at` ni `updated_at`: Eloquent los rellena solo.



CLASE 32 · PUNTO 4

😮 Relaciones

Aquí está la verdadera magia. Las relaciones se definen como **métodos** del modelo y se usan como **propiedades** [3].

Uno a muchos: una categoría tiene muchos cursos

ejemplo.php — php-paso-a-paso/clase32 — Visual Studio Code

ejemplo.php

```php
// Categoria.php
public function cursos(): HasMany
{
    return $this->hasMany(Curso::class);
}

// Curso.php: la otra cara
public function categoria(): BelongsTo
{
    return $this->belongsTo(Categoria::class);
}
```

ejemplo.php — php-paso-a-paso/clase32 — Visual Studio Code

ejemplo.php

```php
$curso->categoria->nombre;              // "Programación": Eloquent hace la consulta por nosotros
$categoria->cursos;                     // colección con sus cursos
$categoria->cursos()->where('horas', '>', 40)->get();   // con () es una consulta: se puede filtrar
$categoria->cursos()->create([...]);    // crea un curso ya unido a la categoría
```

Eloquent sabe qué columnas unir **por las convenciones**: el `belongsTo` de `categoria()` busca la columna `categoria_id`. Si la hubiéramos llamado distinto, habría que decírselo: `belongsTo(Categoria::class, 'mi_columna')`.

Muchos a muchos: alumnos y cursos

ejemplo.php — php-paso-a-paso/clase32 — Visual Studio Code

ejemplo.php

```php
// Curso.php
public function alumnos(): BelongsToMany
{
    return $this->belongsToMany(Alumno::class)
        ->withPivot('nota')       // columnas extra de la tabla intermedia
        ->withTimestamps();       // y sus fechas
}

// Alumno.php
public function cursos(): BelongsToMany
{
    return $this->belongsToMany(Curso::class)
        ->withPivot('nota')
        ->withTimestamps();
}
```

La tabla intermedia `alumno_curso` se deduce de los dos nombres en orden alfabético (clase 31). Y sus columnas se leen a través de `pivot`:

ejemplo.php — php-paso-a-paso/clase32 — Visual Studio Code

ejemplo.php

```php
foreach ($curso->alumnos as $alumno) {
    echo $alumno->nombre . ': ' . $alumno->pivot->nota;
}
```

Para matricular y desmatricular:

ejemplo.php — php-paso-a-paso/clase32 — Visual Studio Code

ejemplo.php

```php
$alumno->cursos()->attach(3);                    // matricula en el curso 3
$alumno->cursos()->attach(3, ['nota' => 8.5]);   // con nota
$alumno->cursos()->detach(3);                    // lo quita
$alumno->cursos()->sync([1, 3, 5]);              // deja exactamente esos tres
$alumno->cursos()->updateExistingPivot(3, ['nota' => 9]);
```

Así ha quedado el `AlumnoSeeder`, mucho más claro que con `DB::table()`:

ejemplo.php — php-paso-a-paso/clase32 — Visual Studio Code

ejemplo.php

```php
$cursos = Curso::pluck('id');

// Cada alumno se matricula en entre 1 y 3 cursos distintos
Alumno::factory(40)->create()->each(function (Alumno $alumno) use ($cursos) {
    foreach ($cursos->random(rand(1, 3)) as $cursoId) {
        $alumno->cursos()->attach($cursoId, [
            'nota' => fake()->optional(0.6)->randomFloat(2, 3, 10),
        ]);
    }
});
```

Uno a uno: un usuario tiene un perfil

La tabla `perfiles` tiene una columna `user_id` **única** (migración), y la relación es un `hasOne`:

ejemplo.php — php-paso-a-paso/clase32 — Visual Studio Code

ejemplo.php

```php
// User.php
public function perfil(): HasOne
{
    return $this->hasOne(Perfil::class);
}
```

ejemplo.php — php-paso-a-paso/clase32 — Visual Studio Code

ejemplo.php

```php
$ramon->perfil()->create(['ciudad' => 'Santander', 'biografia' => '...']);
$ramon->perfil->ciudad;      // "Santander"
$perfil->user->name;         // y al revés, con belongsTo
```

Relación	En un lado	En el otro	Ejemplo
Uno a uno	`hasOne`	`belongsTo`	Usuario → perfil
Uno a muchos	`hasMany`	`belongsTo`	Categoría → cursos
Muchos a muchos	`belongsToMany`	`belongsToMany`	Alumnos ↔ cursos



💡 **Truco para no liarse:** `belongsTo` va siempre en el modelo cuya tabla **tiene la columna** `xxx_id`.



CLASE 32 · PUNTO 5

⚠️ Carga ansiosa y el problema N+1

Mirad este código, que parece inocente:

ejemplo.php — php-paso-a-paso/clase32 — Visual Studio Code

ejemplo.php

```php
$cursos = Curso::all();                 // 1 consulta

foreach ($cursos as $curso) {
    echo $curso->categoria->nombre;     // ¡1 consulta MÁS por cada curso!
}
```

Con 15 cursos, son **16 consultas**. Con 1.000, son 1.001. Es el famoso **problema N+1**, y es el error de rendimiento más típico con un ORM [4]. La solución es la **carga ansiosa** (*eager loading*): pedir las relaciones **por adelantado** con `with()`:

ejemplo.php — php-paso-a-paso/clase32 — Visual Studio Code

ejemplo.php

```php
$cursos = Curso::with('categoria')->get();   // 2 consultas, haya los cursos que haya
```

Eloquent hace una consulta para los cursos y **otra** para todas sus categorías (`WHERE id IN (1, 2, 3)`), y las reparte.

ejemplo.php — php-paso-a-paso/clase32 — Visual Studio Code

ejemplo.php

```php
Curso::with(['categoria', 'alumnos'])->get();       // varias relaciones
Alumno::with('cursos.categoria')->find(3);          // relaciones anidadas
$curso->load('alumnos');                            // cargar después, sobre un objeto que
ya tenemos
Curso::withCount('alumnos')->get();                 // solo contar: añade $curso
->alumnos_count
```

Para que no se nos escape nunca, en `AppServiceProvider` le pedimos a Laravel que, mientras desarrollamos, **lance un error** si cargamos una relación sin `with()`:

ejemplo.php — php-paso-a-paso/clase32 — Visual Studio Code

ejemplo.php

```php
public function boot(): void
{
    // En desarrollo, avisa con un error si cargamos relaciones sin with() (problema N+1)
    Model::preventLazyLoading(! app()->isProduction());

    // Los enlaces de la paginación, con un HTML sencillo que estilamos en academia.css
    Paginator::useBootstrapFour();
}
```

Si os sale una `LazyLoadingViolationException`, ya sabéis: os falta un `with()` o un `load()`.



CLASE 32 · PUNTO 6

🔍 Scopes

¿Cuántas veces vamos a escribir `where('publicado', true)`? En la portada, en el listado, en las categorías, en la API... Un ***scope*** le pone **nombre** a un trozo de consulta [5]:

ejemplo.php — php-paso-a-paso/clase32 — Visual Studio Code

ejemplo.php

```php
// Curso.php
#[Scope]
protected function publicados(Builder $consulta): void
{
    $consulta->where('publicado', true);
}

#[Scope]
protected function buscar(Builder $consulta, string $texto): void
{
    $consulta->where('titulo', 'like', "%{$texto}%");
}
```

Y se usan como si fueran métodos del modelo, encadenados con el resto:

ejemplo.php — php-paso-a-paso/clase32 — Visual Studio Code

ejemplo.php

```php
Curso::publicados()->get();
Curso::publicados()->buscar('php')->orderBy('titulo')->get();
$categoria->cursos()->publicados()->count();
```

Si mañana «publicado» significa además «con fecha de inicio», lo cambiamos en **un sitio**.



CLASE 32 · PUNTO 7

📘 Casts, accessors y mutators

Casts: convertir tipos

De la base de datos todo llega como texto o número. Los ***casts*** le dicen a Eloquent cómo convertir cada columna [6]:

ejemplo.php — php-paso-a-paso/clase32 — Visual Studio Code

ejemplo.php

```php
protected function casts(): array
{
    return [
        'precio' => 'decimal:2',      // "390.00"
        'fecha_inicio' => 'date',     // un objeto Carbon
        'publicado' => 'boolean',     // true / false, en vez de 1 / 0
    ];
}
```

Gracias al *cast* `date`, en la vista podemos escribir `$curso->fecha_inicio->format('d/m/Y')`, o incluso `$alumno->fecha_nacimiento->age` para la edad. Otros *casts* útiles: `'integer'`, `'array'` (guarda un array como JSON), `'hashed'` (cifra la contraseña, lo veréis en `User`).

Accessors: atributos calculados

Un ***accessor*** crea un atributo que **no existe** en la tabla, o transforma uno que sí existe al **leerlo**:

ejemplo.php — php-paso-a-paso/clase32 — Visual Studio Code

ejemplo.php

```php
// $curso->precio_con_iva
protected function precioConIva(): Attribute
{
    return Attribute::make(
        get: fn () => round($this->precio * (1 + self::IVA), 2),
    );
}

// $alumno->nombre_completo
protected function nombreCompleto(): Attribute
{
    return Attribute::make(
        get: fn () => "{$this->nombre} {$this->apellidos}",
    );
}
```

El método se llama en `camelCase` (`precioConIva`) y se usa en `snake_case` (`precio_con_iva`), como una columna más.

Mutators: transformar al guardar

Un ***mutator*** transforma el valor **al asignarlo**. El correo del alumno siempre se guardará en minúsculas, y el título del curso genera además su `slug`:

ejemplo.php — php-paso-a-paso/clase32 — Visual Studio Code

ejemplo.php

```php
// Alumno.php
protected function email(): Attribute
{
    return Attribute::make(
        set: fn (string $valor) => mb_strtolower(trim($valor)),
    );
}

// Curso.php: devolviendo un array, rellena varias columnas
protected function titulo(): Attribute
{
    return Attribute::make(
        set: fn (string $valor) => [
            'titulo' => trim($valor),
            'slug' => Str::slug($valor),
        ],
    );
}
```

ejemplo.php — php-paso-a-paso/clase32 — Visual Studio Code

ejemplo.php

```php
$curso = Curso::create(['categoria_id' => 1, 'titulo' => '  Vue.js desde cero ', 'horas' => 40, 'precio' => 200]);
$curso->titulo;           // "Vue.js desde cero"
$curso->slug;             // "vuejs-desde-cero"
$curso->precio_con_iva;   // 242
```



CLASE 32 · PUNTO 8

🤔 Borrado lógico

En una academia real, ¿se borra un curso que ya se impartió? Mejor no: perderíamos el historial de matrículas y notas. El **borrado lógico** (*soft delete*) no borra la fila: le pone fecha a una columna `deleted_at`, y a partir de ahí Eloquent hace como si no existiera [7].

Necesitamos la columna (migración):

ejemplo.php — php-paso-a-paso/clase32 — Visual Studio Code

ejemplo.php

```php
Schema::table('cursos', function (Blueprint $table) {
    // La columna deleted_at que necesita el borrado lógico
    $table->softDeletes();
});
```

Y el *trait* en el modelo:

ejemplo.php — php-paso-a-paso/clase32 — Visual Studio Code

ejemplo.php

```php
class Curso extends Model
{
    use HasFactory;
    use SoftDeletes;
```

ejemplo.php — php-paso-a-paso/clase32 — Visual Studio Code

ejemplo.php

```php
$curso->delete();                       // no borra: rellena deleted_at
Curso::count();                         // 14: ya no lo cuenta
Curso::withTrashed()->count();          // 15: incluyendo los borrados
Curso::onlyTrashed()->get();            // solo los borrados (la «papelera»)
Curso::withTrashed()->find(5)->restore();   // lo recupera
$curso->forceDelete();                  // ahora sí, borrado de verdad
```



CLASE 32 · PUNTO 9

👌 Paginación

Con 15 cursos todo cabe en una página, pero con 500... Cambiar `get()` por `paginate()` es todo lo que hace falta [8]:

ejemplo.php — php-paso-a-paso/clase32 — Visual Studio Code

ejemplo.php

```php
$cursos = Curso::publicados()->orderBy('titulo')->paginate(6);
```

Laravel lee solo el `?page=` de la URL, calcula el `LIMIT` y el `OFFSET` y cuenta el total. En la vista:

plantilla.php — php-paso-a-paso/clase32 — Visual Studio Code

plantilla.php

```
<p>Mostrando del {{ $cursos->firstItem() }} al {{ $cursos->lastItem() }} de {{ $cursos
->total() }} cursos</p>

@foreach ($cursos as $curso) ... @endforeach

{{ $cursos->links() }}
```

`links()` pinta los enlaces a las páginas. Por defecto usa clases de Tailwind; como nosotros no lo usamos, en `AppServiceProvider` elegimos el HTML de Bootstrap 4 (`Paginator::useBootstrapFour()`), que es una lista sencilla que estilamos en `academia.css`. Los textos «Anterior» y «Siguiente» salen de `lang/es/pagination.php`.



⚠️ Si la página tiene filtros (`?buscar=php`), añadid `->withQueryString()`: si no, al pasar a la página 2 se pierde la búsqueda.



CLASE 32 · PUNTO 10

🚀 La academia con Eloquent

Comparad el `CursoController` de hoy con el de la clase 31 (ver fichero):

ejemplo.php — php-paso-a-paso/clase32 — Visual Studio Code

ejemplo.php

```php
public function index(Request $request): View
{
    $buscar = $request->query('buscar', '');

    $cursos = Curso::publicados()
        ->with('categoria')          // carga ansiosa: evita el problema N+1
        ->withCount('alumnos')       // añade $curso->alumnos_count
        ->when($buscar !== '', fn (Builder $consulta) => $consulta->buscar($buscar))
        ->orderBy('titulo')
        ->paginate(6)
        ->withQueryString();         // los enlaces de página conservan ?buscar=

    return view('cursos.index', [
        'cursos' => $cursos,
        'buscar' => $buscar,
    ]);
}

// GET /cursos/{curso}: route model binding, o 404 si no existe
public function show(Curso $curso): View
{
    $curso->load(['categoria', 'alumnos' => fn ($consulta) => $consulta
->orderBy('apellidos')]);

    return view('cursos.show', ['curso' => $curso]);
}
```

¿Recordáis el *route model binding* de la clase 29? Ya está aquí: `show(Curso $curso)` recibe el curso cargado o responde con un 404. Y en la vista, `route('cursos.show', $curso)` acepta el objeto directamente.

Binding por otra columna

Las categorías las buscamos por su `slug`, que queda mucho mejor en la URL (`/categorias/programacion`). Basta con indicarlo en la ruta:

ejemplo.php — php-paso-a-paso/clase32 — Visual Studio Code

ejemplo.php

```php
Route::get('/categorias/{categoria:slug}', [CategoriaController::class, 'show'])
->name('categorias.show');
```

ejemplo.php — php-paso-a-paso/clase32 — Visual Studio Code

ejemplo.php

```php
public function show(Categoria $categoria): View
{
    $cursos = $categoria->cursos()
        ->publicados()
        ->with('categoria')
        ->withCount('alumnos')
        ->orderBy('titulo')
        ->paginate(6);

    return view('categorias.show', [
        'categoria' => $categoria,
        'cursos' => $cursos,
    ]);
}
```

Y la ficha del curso enseña ahora sus alumnos con la fecha de matrícula y la nota, sacadas de `pivot` (`cursos/show.blade.php`):

ejemplo.php — php-paso-a-paso/clase32 — Visual Studio Code

ejemplo.php

```blade
@forelse ($curso->alumnos as $alumno)
    <tr>
        <td>{{ $alumno->nombre_completo }}</td>
        <td>{{ $alumno->pivot->created_at->format('d/m/Y') }}</td>
        <td>{{ $alumno->pivot->nota !== null ? number_format($alumno->pivot->nota, 2, ',', '.') : '—' }}</td>
    </tr>
@empty
    <tr><td colspan="3">Todavía no hay nadie matriculado.</td></tr>
@endforelse
```

Nuestro `Modelo` (clase 25)	Eloquent
`Curso::todos()`	`Curso::all()`
`Curso::buscar($id)`	`Curso::find($id)`
`Curso::paginaDonde('titulo', $texto, $pagina, 6)`	`Curso::where('titulo', 'like', ...)->paginate(6)`
`$curso->rellenar($datos)`	`$curso->fill($datos)` (con `#[Fillable]`)
`$curso->guardar()`	`$curso->save()`
`$curso->borrar()`	`$curso->delete()`
`protected static string $tabla`	`#[Table('...')]` (solo si no sigue la convención)
`protected static array $columnas` con los tipos	`casts()`
No teníamos relaciones	`hasOne`, `hasMany`, `belongsTo`, `belongsToMany`



Las soluciones se copian dentro del proyecto, respetando sus carpetas.

- ☐ **01 · Los alumnos.** Cread `AlumnoController` con `index` (tabla paginada de 10 en 10, ordenada por apellidos, con el número de cursos de cada uno gracias a `withCount`) y `show` (sus cursos con la categoría y la nota, y la nota media). → `AlumnoController.php`, `vistas` y `routes/web.php`
- ☐ **02 · Filtrar por precio.** Cread el *scope* `precioMaximo($maximo)` en `Curso` y añadid al listado un desplegable «Hasta 150 / 250 / 350 €» que filtre los cursos (`?max=250`) sin perder la búsqueda ni la paginación. → `Curso.php`, `CursoController.php` y `cursos/index.blade.php`
- ☐ **03 · Últimas plazas.** Cread el *accessor* `plazas_libres` (plazas menos alumnos matriculados) y mostrad en la tarjeta «¡Últimas N plazas!» cuando queden 3 o menos, y «Completo» cuando no quede ninguna. → `Curso.php` y `tarjeta-curso.blade.php`
- ☐ **04 · Dos recuentos de la misma relación.** Una relación `cursosPublicados()` en `Categoria` y una página `/categorias` con el total de cursos y los publicados de cada una. Pista: `withCount(['cursos', 'cursos as publicados_count' => fn ($consulta) => ...])`. → `Categoria.php`, `CategoriaController.php` y `categorias/index.blade.php`
- ☐ **05 · Buscar o crear.** `php artisan academia:categoria "Idiomas"` crea la categoría solo si no existe, con `firstOrCreate()`, y dice si la ha creado o ya estaba (`wasRecentlyCreated`). → `routes/console.php`
- ☐ **06 · Filtrar por una relación.** Un *scope* `deCategoria($slug)` en `Curso` que use `whereHas()`, para que el listado acepte `?categoria=diseno`. → `Curso.php` y `CursoController.php`
- ☐ **07 · Los alumnos de una categoría.** `/categorias/{slug}/alumnos`: los alumnos con algún curso de esa categoría y, para cada uno, solo esos cursos. Pista: `whereHas()` para filtrar y `with()` con condiciones para cargar. → `CategoriaController.php` y `categorias/alumnos.blade.php`

- ☐ **08 · Slug automático.** Un *mutator* en `Categoria` que, al asignar el nombre, rellene también el `slug`, como el del título del curso. → `Categoria.php`
- ☐ **09 · La papelera por consola.** `php artisan academia:papelera 5` manda un curso a la papelera; con `--restaurar` lo recupera y con `--forzar` lo borra para siempre. Pista: `withTrashed()`. → `routes/console.php`
- ☐ **10 · Colecciones.** `php artisan academia:resumen`: cursos, precio medio y el más caro de cada categoría, sin escribir SQL, con los métodos de las colecciones (`groupBy`, `map`, `avg`, `sortByDesc`...). → `routes/console.php`



🏆 **Reto:** cread una página `/papelera` que liste los cursos borrados (`onlyTrashed()`) con un botón para restaurar cada uno. Pista: el botón tiene que ser un formulario `POST`; si os atascáis, esperad a la clase 33, donde veremos los formularios en Laravel.



CLASE 32 · PUNTO 12

👋 Lo que tenéis que llevaros de hoy

Idea	En una frase
Eloquent	Cada tabla es un modelo y cada fila, un objeto
Convenciones	Modelo en singular, tabla en plural; si no, `#[Table('...')]`
`#[Fillable]`	Qué columnas se pueden rellenar con un array
`find`, `where`, `create`, `update`, `delete`	El CRUD sin escribir SQL
`hasOne`, `hasMany`, `belongsTo`, `belongsToMany`	Las relaciones, usadas como propiedades
`pivot`, `attach`, `sync`	La tabla intermedia y las matrículas
`with()` / `withCount()`	Carga ansiosa: adiós al problema N+1
`#[Scope]`	Trozos de consulta con nombre: `Curso::publicados()`
Casts	Tipos correctos: fechas Carbon, booleanos, decimales
Accessors / *mutators*	Atributos calculados / transformar al guardar
`SoftDeletes`	Borrar sin perder: `deleted_at`, `restore()`
`paginate()` + `links()`	Paginación en dos líneas (con `withQueryString()`)

📚 Fuentes y documentación oficial

1. Laravel: Eloquent, Getting Started
2. Laravel: Eloquent, Retrieving Models
3. Laravel: Eloquent Relationships
4. Laravel: Eager Loading
5. Laravel: Query Scopes
6. Laravel: Eloquent Mutators and Casting
7. Laravel: Soft Deleting
8. Laravel: Pagination